

Privacy-preserving federated learning in asynchronous environment using homomorphic encryption

Mansi Gupta^a, Mohit Kumar^a,^{*}, Renu Dhir^b

^a Department of Information Technology, Dr BR Ambedkar National Institute of Technology, Jalandhar, India

^b Department of Computer Science and Engineering, Dr BR Ambedkar National Institute of Technology, Jalandhar, India

ARTICLE INFO

Keywords:

Federated learning
Asynchronous
Privacy-preserving
Homomorphic encryption

ABSTRACT

Integrating Federated Learning (FL) into Internet of Things (IoT)-based smart systems has introduced as a promising approach for enhancing data privacy by allowing decentralized model training. However, even with FL, the risk of privacy breaches persists, especially during communication phases, where adversaries may perform reverse engineering on gradients to infer sensitive information. To mitigate this, cryptographic techniques such as Secure Multiparty Computation (SMPC), Homomorphic Encryption (HE), and Differential Privacy (DP) are commonly employed. This paper proposes a novel FL framework that combines an improved Paillier HE scheme with quantization-based compression to enhance privacy, reduce communication overhead, and ensure computational efficiency. The framework operates in asynchronous settings by employing a dynamic timestamp mechanism to synchronize client updates and facilitate an efficient aggregation. The experimental evaluation using MNIST, CIFAR-10, and Fashion-MNIST datasets showed that the proposed method reduced the total training time and communication cost by 12.46% (for MNIST and CIFAR-10) and 14.72% (for Fashion-MNIST), respectively, compared to baseline models. The results confirmed the effectiveness of our approach in safeguarding privacy while maintaining scalability and resource efficiency in real-world IoT applications.

1. Introduction

Artificial Intelligence (AI) enables machines or devices to analyze human reasoning and learning abilities [1]. It has a significant role in the domain of medical, transportation, smart homes, and agriculture [2]. In AI, the collection of data is important as it requires a certain amount of data to train the model for reliable AI-based services. Moreover, the performance of the system directly relies on the quality of training data. Moreover, there is a high possibility of data leakage for private or sensitive data, which in turn, clients or devices hesitate to share data [3]. To prevail over these challenges, FL presented by Google comes up as a promising solution [4]. FL enables devices to train an ML model collaboratively without leaking their sensitive data. In FL, server or centralized party transmits a global model to selected clients, who return local model updates to server when local training completes. Lastly, the third party or central server aggregates received local model updates and transmits back the modified global model parameter, this procedure iterates till the optimal performance achieved [5]. Nevertheless, despite utilizing FL, still remains a probability of privacy breaches and data leakage, specifically during communication, where intruder uses reverse engineering to gain raw data from gradients [6].

FL utilizes privacy preservation techniques in order to generate secure data communication or model training. In this paper, we utilize Homomorphic Encryption (HE) as an added solution. HE enables arithmetic operations such as multiplication and addition on ciphertext without decryption. Although, implementing HE in FL scheme ensures privacy, while adding the cost to communication and computation overhead. The challenges faced by HE-based FL systems are described as below:-

- **Stragglers:** Recent studies done on HE-based FL fail to account for resource or data heterogeneity, and presuming that clients are available anytime, ignoring the fact of stragglers which creates delays.

Moreover, the time during encryption process further increases the server idle state and waste resources. Fig. 1 illustrates iteration time breakdowns for FMNIST and CIFAR on both client and server sides. Clients primarily spend time on encryption and idle states, with moderate computation and decryption. Servers remain mostly idle, followed by dispatch, aggregation, and decryption. In this environment, the FL process convergence rate is proportional to the speed of slowest device is called lag effect. To solve the problem of stragglers, asynchronous FL is

* Corresponding author.

E-mail address: kumarmohit@nitj.ac.in (M. Kumar).

Table 1
Comparison between synchronous and asynchronous FL.

Characteristics	Synchronous FL	Asynchronous FL
Aggregation frequency	Low	High
Stragglers	Highly affected	Relatively unaffected
Fault tolerance	Less tolerant; missing updates delayed aggregation	More tolerant; system pursue with available updates
Model consistency	Updates are synchronized	Cause potential staleness in some updates
Communication overhead	Higher	Lower
Resource utilization	Low; fast clients may need to wait for slow clients	High; clients contribute as they become ready
Use cases	Ideal for settings where customers exhibit comparable levels of performance and dependability	Best for heterogeneous environments

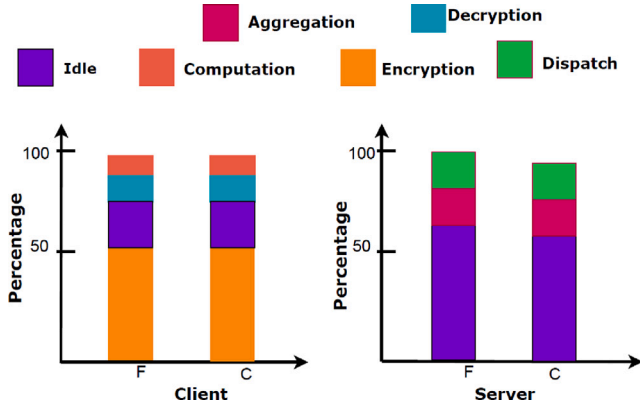


Fig. 1. Iteration time breakdowns of FMNIST (on the left side) and CIFAR (on the right side) for a client and the server.

introduced, where server immediately updates the parameters whenever they received. The comparison between synchronous and asynchronous FL is presented in Table 1

- **Communication Overhead:** Encryption procedure increases data size significantly proportionate to the key size used, which in turn, maximizes network bandwidth utilization or communication cost. Larger the key size, the greater encryption and overall training time takes.

This manuscript presents a FL framework that integrates improved Paillier homomorphic encryption and quantization-based compression to address challenges related to critical privacy, security, and communication efficiency. Our approach is tailored for asynchronous settings, where clients may operate on different schedules and submit updates at varying times. The iterative process is managed with a timestamp mechanism to synchronize client updates and ensure timely aggregation, which further improves system idleness, reducing training time. Our approach effectively balances privacy protection, computational efficiency, and communication cost.

1.1. Motivation & Contributions

Federated learning (FL) preserves data privacy and lowers transmission costs by allowing several clients or device to train a shared model cooperatively without centralizing their data. However, FL has issues, particularly with data security and the efficacy of model updates. The possibility of data reconstruction attacks as shown in Fig. 2, in which malicious users can deduce sensitive information from the shared model updates, is a significant problem in FL. This issue emphasizes the need for better security measures to protect client information during training.

Homomorphic encryption (HE) is an emerging powerful solution to this challenge. It enables computations on encrypted data, HE makes sure that no external parties, not even the central server, may access the raw data. This characteristic makes HE a great aid for protecting FL procedures. However, classic FL implementations with HE frequently

work synchronously, requiring all clients to submit their updates at the same time before aggregation can take place. This synchronous nature introduces several issues in terms of, inefficiency and delays, scalability, and resource utilization. The server must wait for updates from all clients, which may cause delays if some have slower network connections or computational capacity. To overcome these concerns, an asynchronous FL architecture based on homomorphic encryption is required. An asynchronous technique enables clients to transmit encrypted updates depending on their own availability, without waiting for other clients. This approach guarantees improved resource management and ongoing model training advancement. As far as our knowledge, we are first to utilize the HE-based FL mechanism in an asynchronous environment.

- A novel asynchronous FL scheme utilizing improved Paillier homomorphic encryption is proposed, which utilizes an incremental timestamp-based weighted average algorithm to handle the lag effect, where the initial timestamp is justified and analyzed.
- A communication-efficient procedure, Quantization, is utilized to handle large-size ciphertext packets transmitted between clients and servers.
- It verifies that the model trained by our proposed algorithm has similar performance metrics, such as accuracy and time reduction, in comparison to the model trained by traditional methods implementing HE.
- Regarding homomorphic encryption's overhead, the manuscript examines the effects of varying key lengths and concludes that the overhead rises with key length. Additionally, security analysis is carried out.

1.2. Organization

The rest of the manuscript is organized as follows. Section 2 details an overview of the related work. Section 3 presents the prerequisites correlated to FL and privacy-preserving techniques. Section 4 outlines the proposed schemes with steps performed generalizing key generation, distribution, local training, and aggregation. Section 5 presents a theoretical security analysis of the proposed scheme. Experimental results are represented in Section 6 to verify proposed scheme performance. Future scopes and result analysis are described in Section 7. Finally, Section 8 concludes the paper with the references at the end. The notations used in the entire manuscript is shown in Table 2.

2. Related work

The comparison of existing approaches in terms of encryption mechanism, key management policies whether the single or multi-key structure is utilized, aggregation of model updates performed on encrypted or decrypted data, the dataset used for evaluation purposes with the testing environment, experimental results obtained, and their limitations can be shown in Table 3.

Hanchao Ku et al. [7] proposed a privacy-preserving FL scheme with Homomorphic re-encryption, assuring efficiency and data security in medical diagnosis. The three-tier approach consists of IoT devices,

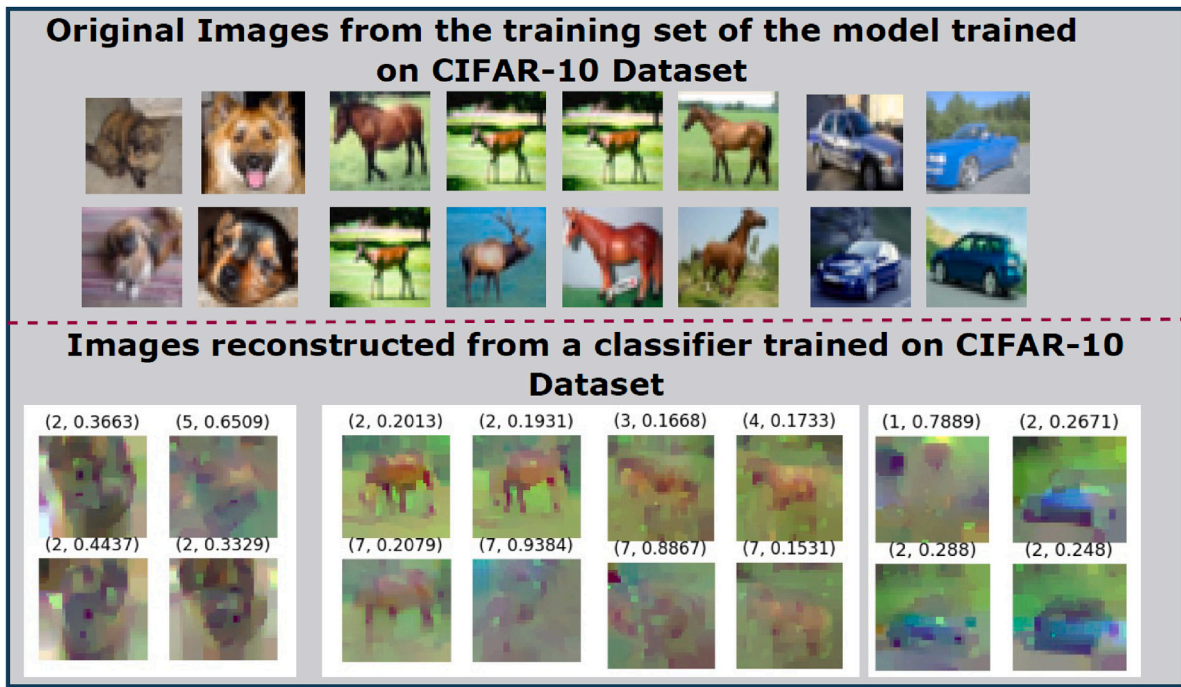


Fig. 2. Data Reconstruction attack performed on CIFAR-10 Dataset for 50 iterations.

Table 2

Notations used in the manuscript with their definitions.

Notation	Definition
N	Number of clients
T	Number of training rounds
η	Learning rate
k	Key size (in bits)
τ	Timer
δ	Adaptive timer parameter
P_K	Public key
S_K	Private key
w_0	Initial model parameter
θ	Model gradients
M	Message
C	Cipher text
L	Lagrange function

fog nodes, and servers, where they perform data collection, training, and aggregation, respectively. Jing Ma et al. [8] proposed a Multi-Key Cheon–Kim–Kim–Song (xMK-CKKS) homomorphic encryption scheme to improve privacy protection in distributed ML environments. The method uses Federated Averaging (FedAvg) algorithm and the configurations for the ciphertext modulus, error distribution, RLWE dimension, and key distribution. The noise flooding technique is used to protect secret keys. The suggested xMK-CKKS technique is appropriate for FL applications since it guarantees privacy protection and streamlines decryption procedures.

Jaehyoung Park et al. [9] provide a Privacy-Preserving FL (PPFL) approach that uses an HE scheme to secure learned local model parameters during collaborative machine learning with numerous clients. Key generation entails generating public–private key pairs for users based on huge prime numbers, with the public key computed as the product of primes and private key obtained from the least common multiple of the prime squared. To maintain privacy in the FL process, the Central Server (CS) uses encrypted random vectors to execute homomorphic addition operations on encrypted local model vectors, which are then partially decrypted using partial private keys. The computational expense on the client side is highlighted by performance analysis, which includes extra encryption and decryption procedures to protect local

model parameters and reflect global model updates. The difficulty of encryption and decryption is related to the number of items in the local model vector and the length of the scheme's random number sequence.

Febrianti Wibawa et al. [10] proposed a privacy-preserving convolutional neural network (CNN) for COVID-19 detection utilizing homomorphic encryption and federated learning. The Brakerski–Fan–Vercauteren (BFV) scheme allows homomorphic evaluation while encrypting polynomials. It is utilized due to its ability to manage vast data and its parallel processing capabilities for calculation and efficient data encryption.

Chengliang Zhang et al. [11] proposed BatchCrypt, a system that optimizes safe cross-silo FL by lowering encryption overhead and overall ciphertext volume, resulting in considerable performance gains. The study emphasizes the economic implications of unaddressed communication and computation overhead in HE, emphasizing the importance of efficient solutions such as BatchCrypt in avoiding the excessive Internet data charges and waste of high-end hardware resources in geodistributed data centers. BatchCrypt speeds up training convergence by up to 81 times while reducing total traffic by 101 times, leading to cost savings of up to 99% in cloud settings. Future research might look at BatchCrypt's ability to improve the efficiency of big ML models and its limits when dealing with complex processes such as multiplying ciphertext matrices in Vertical FL.

Haokun Fang et al. [12] use the Multi-party Privacy Protected Machine Learning (PFMLP) technique, which combines FL and HE to safeguard data privacy. Because of its additive homomorphism characteristic, the Paillier encryption method is explicitly employed for homomorphic encryption in PFMLP. The PFMLP algorithm's performance is evaluated experimentally using the MNIST handwriting dataset and metal fatigue strength data. The accuracy difference between the PFMLP-trained and traditionally trained models is less than 1%, demonstrating the efficacy of the privacy-protected machine learning technique.

Li Zhang et al. [13] present a novel federated learning technique that improves data security and confidentiality while maintaining privacy in Internet of Things (IoT)-based healthcare applications. The suggested data-quality-based weighted average approach increases the contribution rate of high-quality local models to global model aggregation, improving convergence and accuracy. The scheme's dropout-tolerable feature allows the federated learning process to continue even

Table 3

Overview of discussed related work with their characteristics.

Ref	Encryption scheme	Key management	Model aggregation	Dataset & implementation details	Results	Limitations
Hanchao Ku et al. [7]	Homomorphic Re-Encryption Scheme (HRES)	Multi-key encryption and use Diffie–Hellman key Exchange	Computing server aggregates the model updates and transmit it to Re-encryption server	UCI dataset/PyCharm	System is robust because of fog nodes	High communication overhead at fog and server level
Jing Ma et al. [8]	MK-CKKS, Paillier	Multi-key encryption	Aggregation performed at the server where decryption can also takes place	UP-FALL dataset/TensorFlow 2.0	Average energy consumption is 2.4 W	Suited for homogeneous environment only
Jaehyoung Park et al. [9]	Additive Homomorphic Encryption	Key-pairs distributed by Key Generation Centre to authenticated clients	Computation providers perform partial decryption and adds average local updates with the random noise then transmit it to the cloud server	Python	Communication overhead is 2.3 times greater than privacy-preserving deep learning	Communication and computational overhead increases exponentially
Febrianti Wibawa et al. [10]	Brakerski/Fan-Vercauteren (BFV) scheme	Same key-pair is used at each client for encryption and decryption	Model aggregation of encrypted data is performed at the central server	COVID-19 dataset/Tensorflow and Microsoft SEAL library	Accuracy, precision, F1 score reaches above 80%	Key can be compromised easily
Chengliang Zhang et al. [11]	Additively homomorphic encryption where Quantization is used for compression	Single key-pair is used for all clients	Batchwise aggregation is performed at the server level	FMNIST, CIFAR, and LSTM/FATE	BatchCrypt achieves training speedup of $23\times - 93\times$, while reducing the communication overhead by $66\times - 101\times$	Computationally expensive to implement on large models
Haokun Fang et al. [12]	Improved Paillier Algorithm	Key-pair handled by key management center	Encrypted gradients added at the server end	MNIST and metal fatigue datasets and tested on Python 3.6 with scikit-learn 0.21.3 and phe 1.4.0.	speed up the training by 25%–28%	Accuracy is low
Li Zhang et al. [13]	ElGamal encryption algorithm while tolerating certain number of client dropouts	Shamir secret sharing and Diffie–Hellman key exchange algorithms utilized	Server aggregates masked updates from receiving atleast certain number of updates from clients	HAM10000 medical dataset/Linux server equipped with Intel Xeon E5-1650 CPU	Attained an accuracy of more than 76.9%.	Suited for homogeneous environment only
Neveen Mohammad Hijazi et al. [14]	FHE scheme in OUCM, OEMC, MUCMs, MECMs approaches	Not mentioned	FedAvg at the server	WUSTL-EHMS-2020, N-BAIOT Dataset/Anaconda Python framework version 3.9.12	OECM, MUCM, and MECM achieved high accuracy, recall, precision, and F1-score (all above 0.997%), whereas OUCM attained an accuracy of 0.911%	High communication overhead

when the number of online clients exceeds a certain threshold, assuring continuous functioning. HAM10000 dataset containing dermatoscopic images of skin lesions is used for evaluation. The suggested system detected lesion cell types with greater than 76.9% accuracy in healthcare applications.

Neveen Mohammad Hijazi et al. [14] focuses on integrating secure federated learning (FL) with fully homomorphic encryption (FHE) to solve security and privacy concerns in IoT communications. The technique outlines a five-phase strategy that includes local model training, user clustering, model weight encryption, local model updates, and global aggregation to enable secure and private FL in IoT contexts utilizing FHE. Four techniques, namely, one unencrypted cluster matrix (OUCM), one encrypted cluster matrix (OECM), many unencrypted cluster matrices (MUCMs), and many encrypted cluster matrices (MECMs), are shown, each with particular steps such as definitions, local training, encryption, global aggregation, and local updates, demonstrating several strategies for securing FL with FHE in IoT environments. Compared to baseline methods, the approaches effectively decrease communication overhead and delay, with gains ranging from 80.15% to 89.98% in communication overhead minimization.

Research gaps identified

The existing frameworks discussed in the field have provided valuable insights, there is still need for additional research and enhancement. To build on these foundations, it is critical to analyze areas that still need to be thoroughly handled or to find possibilities for improvement. The research gaps can be summarized in below points: -

- Integrating Real-Time Systems for Prompt Decision-Making:** The integration of FL with real-time systems is not covered by most frameworks. Real-time processing is essential in applications requiring prompt choices, such as medical diagnostics. They do not encounter the challenges of resource-constrained and asynchronous environments. Additional complexity may arise when managing out-of-sync updates and variable client engagement times in AFL environments. Future work might concentrate on integrating FL with real-time systems to guarantee timely decision-making. It could also optimize decryption techniques to make them less computationally intensive and flexible enough to accommodate the asynchronous nature of client updates. This entails creating methods to manage different latencies for decryption and guaranteeing steady operation even in the event of non-uniform client participation.

- **Communication Overhead:** This refers to the additional resources, time, and bandwidth required for data exchange between components in a distributed system or network beyond the actual data payload. In FL, communication overhead can significantly impact the efficiency and performance of the system, as it consumes computational resources and increases latency, particularly when frequent or large-scale data exchanges are involved. To maximize system scalability and performance, communication overhead must be minimized. While some frameworks try to minimize communication overhead, additional consideration needs to be made to the trade-offs between communication effectiveness and model accuracy. Research may look on performance-and efficiency-balancing adaptive communication techniques.
- **Scalability and Efficiency:** Although BatchCrypt helps to reduce encryption costs, the method's scalability for large-scale applications still needs work. Subsequent studies should look into more effective methods for managing complex models and large-scale deployments, particularly in geo-distributed systems. Additionally, as the size and complexity of models increase, particularly in deep learning applications, the encryption and decryption processes may introduce substantial delays, potentially leading to bottlenecks in the overall system performance. This is particularly critical in asynchronous federated learning environments, where timely aggregation and updates are essential for maintaining model accuracy and convergence rates.

3. Background study

In this section, we emphasize how federated learning poses strict privacy concerns. The analysis of existing techniques for meeting these requirements is shown in Section 3.2

3.1. Federated learning

Federated Learning (FL) enables cooperative learning that allows edge clients to train a global model stored at the central server without exchanging their raw data with each other [4]. In other words, it makes it possible for devices to do local model training and ensure that the raw data stays local. Only information, including local model updates, can be uploaded to a central server, resulting in “available but not visible” data. Suppose there are K clients, each with a local dataset D_n having s number of samples, $D_n = (x_1, y_1), (x_2, y_2), \dots, (x_s, y_s)$ ($1 \leq n \leq K$), and x represents input sample and y denotes respective label. The local training parameters for one client in a single iteration are represented as $\theta_n = \theta_1, \theta_2, \dots, \theta_s$, then the objective function for the FL training process can be defined as:-

$$\min h(w_g) = \frac{1}{K} \sum_{n=1}^K \frac{D_n}{k} L(\theta_n, D_n) \quad (1)$$

where w_g denotes global model parameters, $h(w_g)$ represents a global objective function, and k ($k = \sum_{n=1}^K D_n$) shows the sum of local data samples among all the clients. FL aims to provide security and privacy and follows the listed steps:-

- **Global Model Initialization:** Initializes the hyperparameters for FL, such as global and local number of iterations and learning rate. Then, these hyperparameters and the initial global model w_g^0 are broadcast to all the selected clients.
- **Local Model Training:** In the r th iteration, each local client n where $n \in [1, K]$ retrieves the old global model w_g^{r-1} and calculates updates utilizing optimization techniques like Stochastic Gradient Descent (SGD):

$$\theta_n^{r+1} \leftarrow \theta_n^r - \eta \nabla l(\theta_n^r, D_k) \quad (2)$$

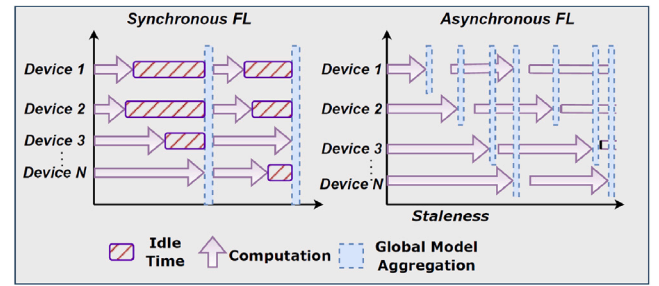


Fig. 3. Outline of synchronous and asynchronous FL framework.

where,

$$\nabla l(\theta_g^t, D_k) = \frac{1}{D_k} \sum_{i=1}^{D_k} \frac{\partial L(h(\theta_g^t, x_i), y_i)}{\partial \theta_g^t} \quad (3)$$

Here, $L(h(\theta_g^t, x_i), y_i)$ denotes loss function and $\nabla l(\theta_g^t, D_k)$ represents local model gradient.

- **Global Model Aggregation:**

Once all clients have completed their training, the central server waits for the updates to aggregate and creates a new global model w_g^{r+1} .

This procedure is iterated up to the maximal number of global rounds or the required degree of accuracy is attained. Based upon the aggregation process, whether to wait for all the clients or not, FL is commonly divided into two categories: synchronous and asynchronous FL. In synchronous FL, model convergence speed depends upon the compatibility of the slowest device; in other words, until all the devices finish their training process, the central server has to stay idle, whereas, in the case of asynchronous FL (AFL), the server will take immediate action upon after receiving parameters even from single client or edge device. The diagrammatic representation of the difference between asynchronous and synchronous FL is given in Fig. 3. While comparing the two techniques, certain factors can be considered. In synchronous FL, there is high communication latency due to stragglers. In contrast, because of high frequency of model updates and exchanges in asynchronous FL, the central server can soon become a communication bottleneck. Each technique has its pros and cons, where the solution to the synchronous approach is proposed in [15–18] and AFL in [19–22].

FL protects privacy by sending local gradients instead of raw data. Still, several research studies indicate that more is needed.[23–25]. There are various threats to FL, including poisoning attacks, inference attacks, Generative Adversarial Network(GAN)-based attacks, backdoor attacks, etc. These listed attacks are highly severe and cause client data manipulation [24]. Poisoning attack [26] has a strong likelihood of happening in FL as every client can access the training data, increasing the chance that tampered data weights will be incorporated to the global ML model. It may happen throughout the training phase and influence the local model or the training dataset, which could affect the accuracy and performance of the global ML model. There are two main categories of poisoning attacks: model and data poisoning attacks. Data poisoning attack is defined as creating false samples for the global model training with the aim to create manipulated model parameters that transmits to the aggregator [27]. Model poisoning involves direct manipulation of the global model by the attacker or malicious client. In contrast, data injection involves inserting fake data into the client's local model processing. Various studies have been conducted, and systems proposed to tackle poisoning attacks [28–32]. Inference attacks exploit the data exchanged during training to deduce sensitive information about the underlying data using some reverse engineering techniques.

Inference attacks are classified as Membership Inference attacks, Attributes Inference Attacks, and Model Inversion Attacks. Malicious

agents use a Membership Inference attack to determine whether a specific data point is related to the training dataset. In contrast, a Model Inversion Attack attempts to recover input data from the model's gradients and outputs. Generative Adversarial Network (GAN)-based attacks in FL can launch poisoning and inference attacks and threaten a given system's privacy and security.

In this article, we are handling data reconstruction attack, in which adversary utilizes reverse engineering to gain personal data from model gradients.

3.2. Privacy preserving techniques

This section covers privacy preservation techniques to mitigate the risk of threats to FL system. These approaches are listed below as:-

Differential Privacy (DP) [33]: It is a popular privacy-preserving technique that hides the real data points or model updates by appending noise to the data. The statistical data quality loss caused by the added noise from each user is disproportionately small in comparison to the improved privacy protection. DP can be defined as per [Definition 1](#):

Definition 1. A randomized function X protects (κ, Δ) -DP given id D_1 and D_2 training datasets differs by only one record, and for all $Y \subset \text{Range}(X)$,

$$Pr[X(D_1) \in Y] < Pr[X(D_2) \in Y] \times e^\kappa + \Delta \quad (4)$$

where Δ denotes failure probability and κ represents privacy budget.

The term $\ln \frac{Pr[X(D_1) \in Y]}{Pr[X(D_2) \in Y]}$ defines privacy loss, where "ln" represents natural logarithm. When $\Delta = 0$, κ -DP is attained.

Secure multi-party computation (SMPC): It is also called as secure function evaluation(SFE) was introduced [34] to protect multi-participant inputs while they were collaboratively computing a function or model. Cryptography is used in SMPC to secure and protect communication. SMPC has been used recently to protect client updates in the FL framework. The objective of multiparty computation (MPC) is to collaboratively compute a function using each party's private input while keeping those inputs secret from other parties. In contrast to the traditional SMPC version, FL's computational performance is greatly enhanced because it only needs to encrypt the parameters rather than a significant amount of data input. The SMPC is a better option in the FL environment because of its performance aspect.

Homomorphic Encryption (HE): This technique was first introduced in 1978 [35] which protects privacy by enabling computations on ciphertext without decrypting it first. An HE scheme F is composed of four main functions:-

$$F = (\text{KeyGeneration}(\text{Gen}), \text{Encryption}(E), \text{Decryption}(D), \text{Evaluation}(\text{Eval}))$$

Suppose E_K represents the encryption function with an encryption key of K . Let P be the plaintext and C be the ciphertext. A cryptosystem is considered homomorphic if it meets the below condition:

$$\forall p_1, p_2 \in P, E_K(p_1 \odot_P p_2) \leftarrow E_K(p_1) \odot_C E_K(p_2) \quad (5)$$

where $\leftarrow$ shows that without intermediary decryption, term on the left side can be computed directly from the term on the right side, or it can be equivalent to it.

Adversarial training: An attacker attempts to fool machine learning models by inserting adversarial samples. Imperfections projected onto real-world data are examples of adversarial data. The adversarial user uses disturbed data to affect the FL model's robustness. To strengthen the FL global model against known attacks, adversarial training is a proactive defense strategy that attempts every possible combination of attacks from the start of the training phase. To deceive the model, adversarial examples – purposefully altered inputs – are added to the

training process during adversarial training. This technique increases the robustness of the model by training it to detect and withstand such threatening events.

To handle data reconstruction attack, HE is studied as a viable solution.

3.3. Homomorphic encryption techniques

It is an encryption technique that takes into account the ciphertext's homomorphic operation nature. It allows clients or users to execute specified algebraic operations on ciphertext directly, and the results of ciphertext computation are identical to those of plaintext encryption following the same operation. Designing an effective HE method requires certain cryptography understanding based on mathematical computation complex theory. Fully homomorphic encryption (FHE) refers to HE algorithms that support any type of ciphertext operation for any number of times. Partial homomorphic encryption (PHE) occurs when just one type of operation, such as multiplication or addition, is enabled for an indefinite number of repetitions. Somewhat Homomorphic Encryption (SHE) permits for multiplication and addition operations on ciphertexts, but only up to a certain complexity. To operate on an encrypted model using the HE approach, all participants must share the same encryption and decryption keys. The classification of HE scheme [36,37] is shown in [Fig. 4](#).

3.3.1. Partial homomorphic encryption (PHE)

This schemes are a kind of encryption techniques that admits for performing specific operations on encrypted data without decrypting it. It accepts either multiplication or addition operations on encrypted data, but not both simultaneously. Examples of PHE schemes include Rivest Shamir Adleman (RSA), introduced by Rivest et al. in 1978 [38], which is an early example of a PHE scheme. PHE schemes have been improved over time by various researchers, such as Goldwasser and Micali in 1982 [39], ElGamal in 1985 [40], Paillier [41], and others, each contributing to enhance the capabilities of PHE. These schemes serve as the foundation for more advanced homomorphic encryption schemes, allowing for secure computations on encrypted data while maintaining privacy. These developments in PHE schemes have played a crucial role in enabling secure data processing and analysis in various applications, ensuring confidentiality and data protection.

3.3.2. Somewhat homomorphic encryption (SHE)

SHE schemes are intermediate solutions between PHE and FHE schemes. SHE schemes support a restricted number of computations or specific circuits, such as branching programs, allowing for more computations than PHE but fewer than FHE. Before 2009, many SHE techniques paved the way for FHE. Early SHE methods like Polly Cracker supported addition and multiplication but suffered from exponential ciphertext expansion. Later systems, such as those proposed by Sander et al. [42] and Ishai and Paskin [43], built on these ideas, although ciphertext sizes grew with operations. The Boneh–Goh–Nissim (BGN) [44] technique improves this by maintaining ciphertext size constant during additions, facilitating FHE development. Despite these advances, many SHE-based FHE attempts remained susceptible to attacks, demonstrating the field's limitations.

3.3.3. Fully homomorphic encryption (FHE)

This schemes allow unlimited arithmetic or algebraic operations on encrypted data, with the resulting output remaining under the ciphertext space. Craig Gentry [45] introduced the first feasible FHE scheme in 2009, addressing a long-standing open problem in cryptography. Due to advanced mathematical concepts, Gentry's ideal lattice-based FHE scheme faced computational cost and complexity challenges, prompting the development of new schemes and optimizations. The primary foundation for the security of novel methods to derive a new FHE scheme is

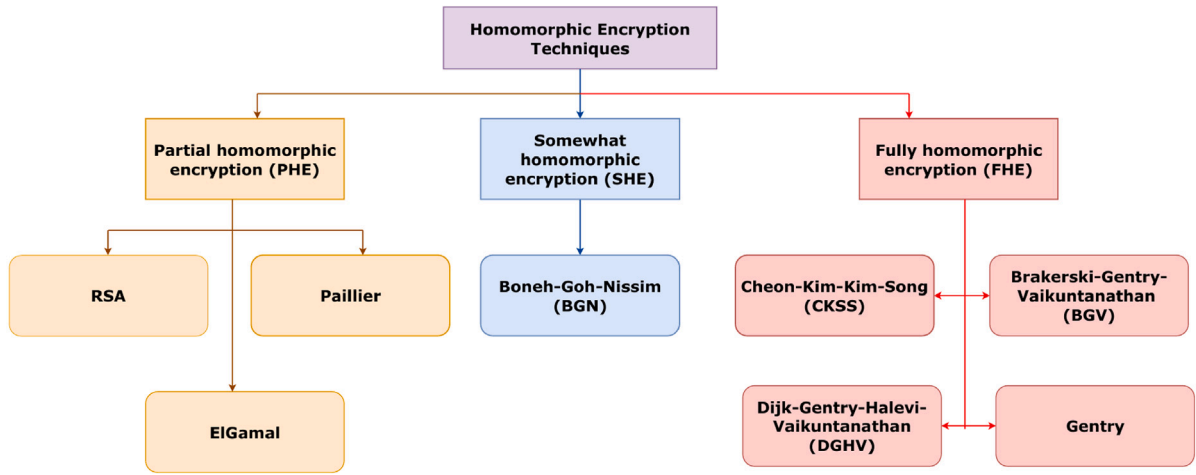


Fig. 4. Categorization of homomorphic encryption techniques.

Table 4
Summary of homomorphic features of different cryptosystems.

Algorithm	Additively homomorphic	Multiplicatively homomorphic	Multiplication with a plain constant	Regeneration of ciphertext	Use cases
RSA		✓	✓	✓	Digital signatures, Secure voting
Paillier	✓		✓	✓	Private information retrieval, Secure voting
ElGamal		✓		✓	Secure messaging, Secure voting
BGN	✓	Limited			Basic statistical calculations
Gentry	✓	✓	✓	✓	Complex encrypted data processing, Cloud computing
DGHV	✓	✓	✓	✓	Complex encrypted data processing, Cloud computing
BGV	✓	✓	✓	✓	Encrypted database queries, Secure cloud computing
CKSS	✓	✓	✓	✓	Machine learning on encrypted data

the complex lattice problem. A lattice(L) is defined as the combination of basis vectors linearly, $a_1, a_2, \dots, a_n$ where L is formulated as:-

$$L = \sum_{j=1}^n \tilde{a}_j * v_j, v_j \in \mathbb{Z} \quad (6)$$

where each vector $a_1, a_2, \dots, a_j$ is known as a basis of the lattice L. FHE schemes have evolved, with different families emerging, including ideal lattice-based, over integers [46], (R)LWE (ring-learning with Error)-based [47], and Nth-degree Truncated Polynomial Ring Units (NTRU)-like schemes [48], each offering unique advantages and challenges. Implementing FHE schemes in real-life applications has been a significant hurdle due to computational costs and mathematical complexities, leading to ongoing research for more efficient solutions.

The comparison between different schemes PHE, SHE, FHE is given in Table 4

4. Privacy-preserved federated learning network framework

The system architecture diagram is presented in Fig. 5. The federated learning framework involves multiple clients collaboratively training a global model while ensuring data privacy and communication efficiency. The workflow started with initializing the FL process. The leader is randomly selected for each training round, who is responsible to generate the public and private key pairs for HE process and also handles the initialization of global model in starting round. The key pairs are generated for each training round differently by varied randomly selected leaders to mitigate the risk of security issues. On the other, an adaptive timer is initialized at the server end when clients starts the training after performing the necessary steps. The clients compress the data before performing encryption, to reduce the encrypted data size, whereas also used batching to group and reduce encryption operations. The server wait for all the clients until the timer expires. After that, perform simple aggregation function and transmit back to all the clients. After receiving the aggregated parameter, clients decrypt

the parameter and iterates the steps until convergence is attained. To ensure system fairness, the timer grows after each training round until it reaches the maximum limit, allowing non-high-performing clients to contribute to the framework. The pseudo code is shown in Algorithm 1 and sequence diagram can be shown in Fig. 6.

The algorithm begins with the selection of a leader from a pool of participating devices. This leader is responsible for initializing the global model and generating the public-private key pair necessary for HE. The global model parameters w_0 , public key (P_K), and private key (S_K) were then distributed to all clients to enable encrypted communication. This architecture ensures that each training round starts with a new cryptographic environment, increasing security by preventing key reuse. On the server side, the training process was structured in rounds. The server randomly selects a leader for each round by calling the function *SELECTLEADER()* and starts timer τ . The leader initiates the training process by executing the key generation and distribution steps. During each round, the server receives encrypted model updates from the clients. However, these updates are accepted only if they arrive before the timer expires. This ensures temporal consistency and prevents stale updates, which could harm the convergence of the global model. Once the timer expires, the server aggregates all the timely received encrypted gradients using the additive homomorphic property of the Paillier cryptosystem, which allows encrypted values to be added without decryption. The aggregated encrypted gradient g_{agg} is then broadcast to all clients. The server then adaptively increments the timer (e.g., $\tau \leftarrow \tau + \delta$) for the next round, allowing flexibility based on the system dynamics and client responsiveness.

On the client side, each client performs local training using its private dataset and the received global model parameters. The resulting local gradient $\Delta\theta'_i$ is then quantized, reducing its numerical precision to lower the communication overhead without significantly affecting model accuracy. The quantized gradients were encrypted using a public key and sent to the server. This approach ensures that sensitive local information is never transmitted in plaintext, thereby maintaining data

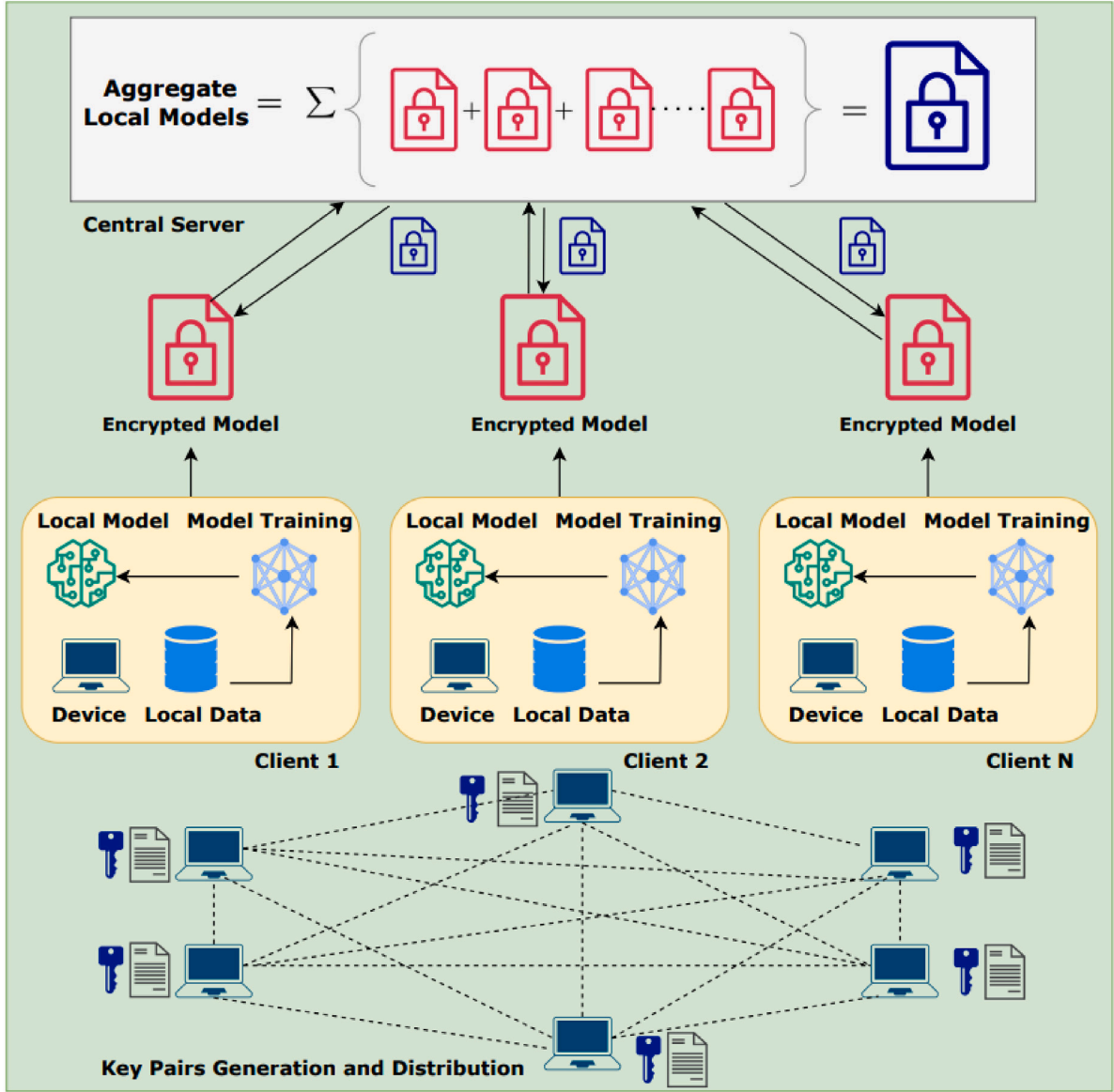


Fig. 5. System architecture of proposed scheme.

privacy throughout training. Once the server broadcasts the aggregated encrypted gradient back to the clients, each client decrypts it using a private key and updates its local model accordingly. Clients asynchronously continued this process, allowing them to operate at their own pace without being held back by slower devices (i.e., stragglers).

The subsequent subsections presents original and improved Paillier HE algorithm with data compression method.

4.1. Paillier algorithm

Paillier Homomorphic Encryption scheme is well-suited for the proposed FL framework because of its additive homomorphic properties, computational efficiency, and compatibility with the system's design objectives. In FL, especially in asynchronous settings, secure aggregation of client updates is crucial. Paillier supports homomorphic addition, allowing the server to aggregate encrypted model updates without decrypting them, thereby matching the core requirement of FL aggregation. Unlike fully homomorphic encryption schemes such as CKKS or BGV, which support addition and multiplication, but are computationally intensive, Paillier offers a lightweight solution ideal for resource-constrained IoT devices. Moreover, the proposed framework

employs quantization-based compression, converting model updates to an integer format to reduce the communication overhead. Paillier operates natively on integers, making it inherently compatible with quantized data, whereas other schemes designed for floating-point arithmetic introduce complexity and noise. In addition, Paillier's non-interactive encryption and decryption processes facilitate asynchronous participation, allowing clients to submit updates independently without real-time coordination. This aligns with the framework's timestamp-driven update mechanism.

As mentioned in the subsequent section, Paillier is a subset of a PHE algorithm that performs addition arithmetic operations. The phases of the algorithm is divided into three sub-categories, as, Key Generation, Message Encryption, and Decryption.

Key Generation: This technique is based on asymmetric key encryption in which both encryption and decryption keys are different. Public keys are shared with all and are used to encrypt the keys, while private keys shared with restricted parties can only decrypt the encrypted message. The steps for generating public and private are numbered as follows:-

- (1) Select any two random large prime numbers p and q that satisfies $\gcd(p * q, (p - 1) * (q - 1)) = 1$

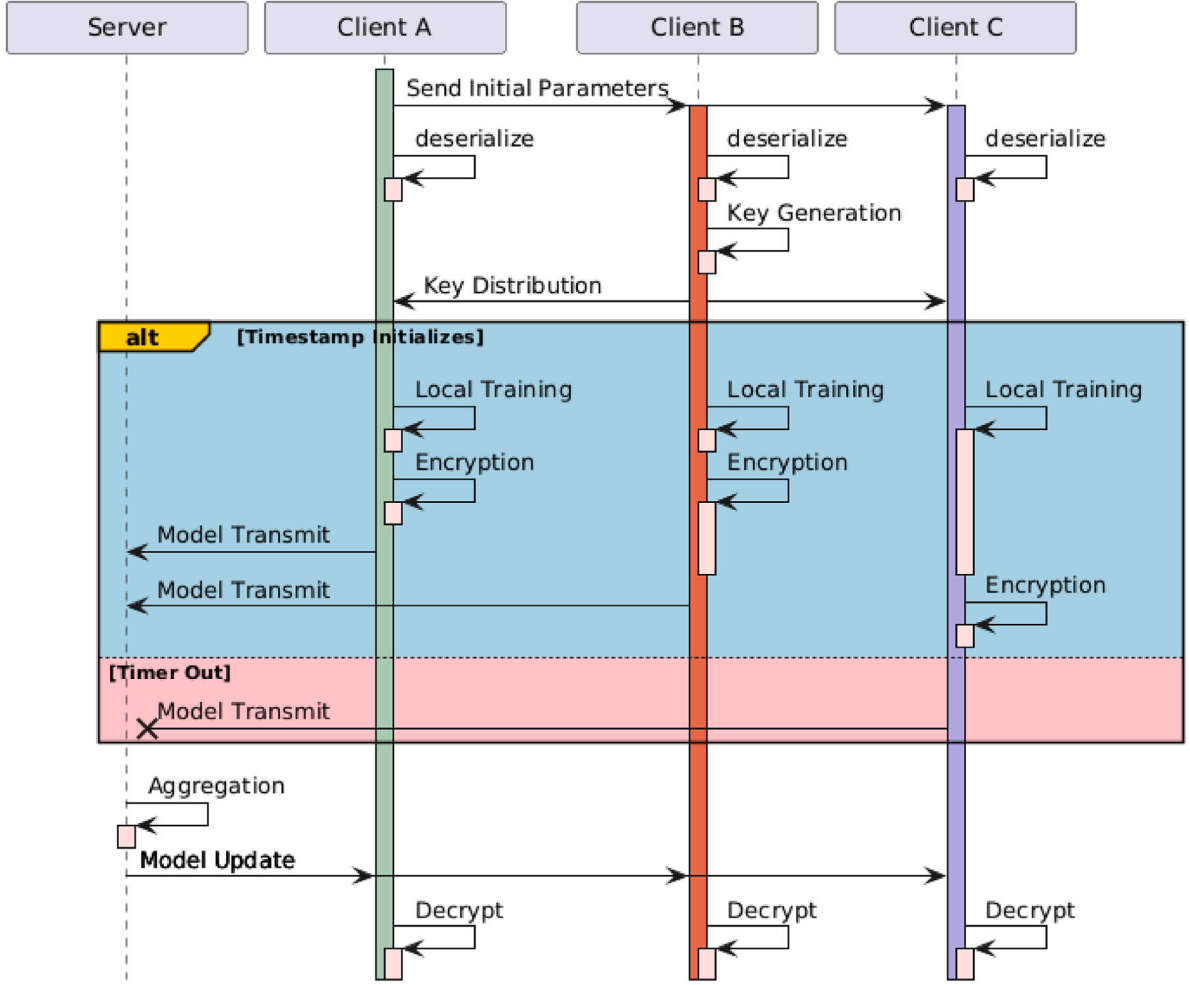


Fig. 6. Working process of asynchronous HE-Based FL using adaptive timestamp strategy.

(2) Compute the modulus n , λ (Carmichael's function) as:-

$$n = p * q \quad (7)$$

$$\lambda = lcm(p-1, q-1) \quad (8)$$

(3) Select random integer g that satisfies $g \in \mathbb{Z}_{n^2}^*$ and n divides the order of g .

(4) Verify that n divides the order of g by determining whether the modular multiplicative inverse of the following exists:

$$\chi = (L(g^\lambda \bmod n^2))^{-1} \bmod n \quad (9)$$

where L is the Lagrange function and calculated as:-

$$L(z) = \frac{(z-1)}{n} \quad (10)$$

The Public Encryption Key (P_K) is (n, g) and Private decryption key (S_K) is (λ, χ)

Message Encryption: Suppose the message or data packet to be encrypted is denoted as M where $M \leq n$ and its ciphertext is denoted as C , find a random number $r(r \in \mathbb{Z}_n^*)$, then the ciphertext can be found as:-

$$C = g^M \cdot r^n \bmod n^2 \quad (11)$$

Message Decryption: Using private key, ciphertext C can be decrypted as:-

$$M = L(C^\lambda \bmod n^2) \cdot \chi \bmod n \quad (12)$$

4.2. Modified paillier algorithm

However, during encryption and decryption, the high complexity of the algorithm will impact the network's training efficiency. Therefore, we proposed a modified version of the Paillier HE algorithm, where the value of g is computed as $g = 1 + \alpha n$ where α must divide n , we need to ensure that g is n th residue modulo n^2 . An integer j is called n th residue modulo n^2 if an integer i exists, such as $i^n \equiv j \bmod n^2$. This modification decreases modular exponentiation & multiplications operations. In the original algorithm, the value of g depends on the context of the group and the elements within it that satisfy the order condition. The encryption and decryption process modifies as:-

Since g becomes $1 + \alpha n$, then the ciphertext C can be computed as:-

$$C = (1 + \alpha n)^M \cdot r^n \bmod n^2 \quad (13)$$

Message can be decrypted using the identical equation as in the Original Paillier algorithm, the proof for the same is described below:-

$$C^\lambda \bmod n^2 = ((1 + \alpha n)^M \cdot r^n)^\lambda \bmod n^2 \quad (14)$$

$$= (1 + \alpha n)^{M\lambda} \cdot r^{n\lambda} \bmod n^2$$

Simplify using the fact $r^{n\lambda}$ that will contribute negligibly, then using Binomial theorem:-

$$(1 + \alpha n)^{M\lambda} = \sum_{j=0}^{M\lambda} \binom{M\lambda}{j} (\alpha n)^j$$

Algorithm 1 Asynchronous HE-based Federated Learning Algorithm

```

function SELECTLEADER()
    Select  $L \in N_1, N_2, \dots, N_N$  ▷ Randomly select a leader from the participating devices
end function
function INITIALIZELEADER()
    Initialize  $w_0$  ▷ Global model initialization
    Generate  $P_K$  and  $S_K$  ▷ HE public and private key pair
    Send  $w_0$ ,  $P_K$ , and  $S_K$  to all other clients
end function

```

Server:

```

for each training round,  $t = 0$  to  $T - 1$  do
    SELECTLEADER()
    State Timer  $t^r_{start}$ 
    INITIALIZELEADER()
    if  $r \leq \tau$  then
        Receive model gradients from the clients ▷ Discard gradient received after timer expires
    end if
     $g^r_{agg} = S \cdot \text{Aggregate}(G_{enc})$ 
    Transmit  $g^r_{agg}$  to all clients
    Increment Timer  $\tau \leftarrow \tau + \delta$  ▷ Until maximum time achieved
end for

```

Client:

```

for each client  $N_i, i \in 1, 2, \dots, N$  do
    Compute local gradient:  $\Delta\theta^r_i$  based on  $w_0$ 
    Quantize gradient:  $\Delta\theta^r_i \rightarrow q(\Delta\theta^r_i)$ 
    Encrypt quantize gradient with  $P_K$ :  $g^r_i = \text{Encrypt}(q(\nabla\theta^r_i), k_i)$ 
    Send encrypted  $g^r_i$  to the server
    Collect  $g^r_{agg}$  from server, and decrypt with  $S_K$ 
    Apply decrypted  $g^r_{agg}$  to local model
end for

```

When reducing modulo n^2 , the terms where $i \geq 2$ will be negligible because $(an)^2$ and higher powers will be multiples of n^2 . Therefore, we focus on the first two terms:

$$(1 + an)^{M\lambda} \equiv 1 + M\lambda an \mod n^2 \quad (15)$$

Substituting Eq. (15) in (14), we get:-

$$C^\lambda \mod n^2 = 1 + \alpha\lambda Mn \mod n^2 \quad (16)$$

After applying Lagrange function(L) in Eq. (16):-

$$L(C^\lambda \mod n^2) = \frac{(1 + anM\lambda) - 1}{n} = \alpha M\lambda \quad (17)$$

$$= \frac{L(C^\lambda \mod n^2)}{\alpha\lambda} \mod n = M \quad (18)$$

Hence proved. The test for additive homomorphism for the modified Paillier algorithm can be performed as:-

$$C_1 = (1 + an)^{M_1} \cdot r_1^n \mod n^2$$

$$C_2 = (1 + an)^{M_2} \cdot r_2^n \mod n^2$$

The sum of ciphertexts C_1 and C_2 is:-

$$C_1 \cdot C_2 = [(1 + an)^{M_1} \cdot r_1^n] \cdot [(1 + an)^{M_2} \cdot r_2^n] \mod n^2$$

$$C_1 \cdot C_2 = (1 + an)^{M_1 + M_2} \cdot (r_1 \cdot r_2)^n \mod n^2 \quad (19)$$

Eq. (19) is a correct encryption of $M_1 + M_2$ with the new randomness $r_1 \cdot r_2$. The decryption of the sum follows:-

$$(C_1 \cdot C_2)^\lambda = (1 + an)^{(M_1 + M_2)\lambda} \cdot (r_1 \cdot r_2)^{n\lambda} \mod n^2 \quad (20)$$

From Eq. (16), we get:-

$$(C_1 \cdot C_2)^\lambda = 1 + an(M_1 + M_2)\lambda \mod n^2 \quad (21)$$

After substituting the Eq. (21) in (17), we achieve:-

$$\equiv M_1 + M_2 \quad (22)$$

By introducing the parameter α and choosing $g = 1 + an$, we have improved the Paillier cryptosystem's efficiency. This change retains the additive homomorphic property and simplifies the decryption process. This novel parametric change provides flexibility and potential performance gains while maintaining the core homomorphic properties of the original Paillier cryptosystem.

4.3. Key distribution mechanism

In the proposed FL framework, a secure key distribution mechanism enables privacy-preserving model updates through HE based on the Paillier cryptosystem. At the beginning of each training round, one client is randomly selected to generate a fresh public-private key pair. The public key is securely broadcast to all participating clients over authenticated and encrypted communication channels, allowing each client to encrypt its local model update prior to transmission to the server.

To facilitate decryption of the aggregated model, the corresponding private key is also securely distributed to all clients. This step is performed using secure transmission protocols and authenticated endpoints to protect against key leakage or unauthorized access. As an additional security enhancement, threshold decryption can be employed to avoid distributing the full private key. In such a scheme, the private key is split among multiple clients, and decryption requires

cooperation from a defined subset, thereby reducing the impact of any single client being compromised, which left for future work.

The key pair is refreshed in every training round, and the responsibility for key generation rotates among clients to prevent reuse and to decentralize trust. Authenticity of the shared keys and messages can be verified using digital signatures or integrity checks to ensure they originate from legitimate sources and remain untampered during transmission.

These measures collectively ensure that model updates remain encrypted and confidential throughout the training process, while supporting asynchronous operation and improving the robustness of the FL system against key compromise and communication-based attacks.

4.4. Data compression mechanism

Including asynchronous processes in federated learning (FL) can greatly speed up the process by providing more flexible and efficient communication between clients and server. Asynchronous approach allow clients to submit encrypted updates without waiting for others, minimizing idle time and increasing system throughput. Even while speeding up HE by itself will not make FL adopt HE easier, what happened if we decreases the data size that required to be encrypted first?. Batching as many gradients as possible to create a large plaintext is reasonable because a mismatch between plaintext and ciphertext lengths mainly causes data inflation. This will significantly minimize the number of encryption operations required. However, the problem remains in maintaining the additive feature of HE after batching without altering ML techniques or reducing accuracy.

Gradient batching appears easy to accomplish. In a Paillier-secured database, batching has actually been utilized to accelerate queries over integers. However, this method is limited to positive integers. The values must be reorganized and grouped based on their exponents to support floating-point numbers. These constraints are crucial for preserving the additivity property of homomorphic encryption (HE) in batched ciphertexts, which requires that the addition of two batched ciphertexts must equivalent to the result of adding the plaintext values in each group element-by-element after decoding.

Gradients are signed floating-point values that must align with their respective model weights and are unable to reordered using exponents. The feasible solution is to utilize an integer representation of the gradients within the batch, which necessitate quantization. In FL, quantization compresses the gradients by converting them from a continuous range to a discrete set of values, thereby reducing the number of bits needed for their representation. Initially, the gradients are normalized to fit inside a given range, like $[-1, 1]$, to use the entire dynamic range of the data. Each gradient value is assigned to the closest accessible quantization level once the normalized gradients have been quantized into discrete levels. The normalization function is shown in Eq. (23).

$$\nabla \theta'_i = \frac{2(\nabla \theta_i - g_{\min})}{g_{\max} - g_{\min}} - 1 \quad (23)$$

Continuous gradient values are transformed into one of q discrete values using q quantization levels. This lowers communication overhead by lowering the data size that needs to be encrypted and transmitted. Quantization results in a considerable reduction in data size, which allows to faster communication and reduced computational burden during encryption, even though it also causes some precision loss. A simple linear quantization function can be shown in Eq. (24)

$$q(\nabla \theta'_i) = \text{round}\left(\frac{\nabla \theta'_i + 1}{2} \times (q - 1)\right) \quad (24)$$

where q is the number of quantization levels and $q(\nabla \theta'_i)$ represents the quantized gradient.

While this work primarily focuses on quantization as a communication-efficient technique, it is also important to compare

it with other popular methods such as gradient sparsification, low-rank approximation, and sketching. Gradient sparsification reduces communication by sending only the most important gradient elements and setting the rest to zero. Although this approach can significantly cut down the amount of data transmitted, it may cause instability in model convergence unless combined with additional error correction techniques. Low-rank approximation compresses gradients by representing them as low-rank matrices, achieving good compression but at the cost of extra computation needed for matrix decomposition. Sketching techniques use random projections to create compressed versions of gradients, offering strong compression guarantees but often requiring careful tuning and adding complexity to the system.

Compared to these alternatives, quantization provides a better balance for our setting. It introduces very little computational overhead, relying mainly on simple arithmetic operations. It also preserves model accuracy across various tasks and is easy to integrate into existing federated learning systems without major changes. Studies in recent literature, such as those by Aji and Heafield (2017) [49] and Alistarh et al. (2017) [50], have shown that quantization is particularly effective when combined with secure aggregation and encryption methods. Because of its simplicity, compatibility with integer-based encryption like homomorphic encryption, and minimal impact on convergence, quantization was selected as the most suitable compression strategy for this work.

5. Security analysis

Our system considers all clients trustable, while the server is semi-honest and curious. Therefore, clients do not tamper with the transmitted message in the communication channel or forge public-private key pairs.

Definition 1 (Decisional Composite Residuosity Assumption (DCRA)). For n th residues modulo n^2 , there exists no polynomial-time distinguisher, i.e. $CR[n]$ is intractable.

Lemma 1. v is a n th residue modulo n^2 if and only if $[[v]]_g = 0$. Furthermore,

$$\forall v_1, v_2 \in Z_{n^2}^* [[v_1 v_2]]_g = [[v_1]]_g + [[v_2]]_g \bmod n$$

that means, for any $g \in B$ the class function $v \rightarrow [[v]]_g$ is a homomorphism from $(Z_{n^2}^*, \times)$ to $(Z_n, +)$ $Class[n, g]$ or n th Residuosity Class Problem of base g , defined as the method of calculating the class function in base g : Calculate $[[v]]_g$ from v , for a given $v \in Z_{n^2}^*$.

Theorem 1. Improved Paillier scheme is secure semantically if and only if the DCRA holds.

Proof. Suppose M_1 and M_2 are two messages and C is the ciphertext of either message M_1 and M_2 . From Lemma 1, $C g^{-M_1 \bmod n^2}$ is n th residue holds if and only if C is the ciphertext of M_1 . As a result, a successfully chosen plaintext attacker can determine composite residuosity. Conversely, if the DCRA holds (i.e., it is hard to decide if a number is an n th residue modulo n^2), then it should also be hard for an attacker to distinguish between the encryptions of M_1 and M_2 . This would mean that the encryption scheme is semantically secure.

6. Experiment & Result analysis

6.1. Evaluation setup

To evaluate the proposed framework, we designed a simulated FL system comprising 03 clients using the Flower framework [51]. In our observations, we select image classification as the training task and employ non-independent and identically distributed (non-IID) training data distributions.

Table 5

Value of constant used during implementation phase.

Parameter	Value
Number of clients	03
Number of epochs	20
Number of federated rounds	20
Key size	128, 256, 512
Batch size	32
Initial learning rate	0.01
Quantization bits	8
initial_timestamp	10
timestamp_increment	5

Federated Datasets: We analyzed the framework using listed three different datasets where two datasets are benchmark datasets, and one is a healthcare dataset to evaluate real-world scenarios while considering non-IID settings

- **CIFAR-10:** The CIFAR-10 [52] dataset is made up of 60,000 images of size 32×32 color splitted into 10,000 test and 50,000 training images which are further partitioned into ten classes, each having 6000 images in number.
- **MNIST:** The dataset [53] contains 60,000 training examples and 10,000 test instances of handwritten images. It is a portion of a more extensive collection accessible from NIST. The digits in the images are size-normalized and centered within a fixed-size frame. The training set is divided into validation sets, as in the CIFAR-10 dataset.
- **Fashion MNIST:** Fashion-MNIST [54] It contains Zalando article photos comprising 60,000 and 10,000 samples for training and test instances, respectively. Each image is 28 pixels in height and width, resulting in a total of 784 pixels per image. A single pixel value represents darkness or lightness; where larger values denotes darker pixels. The pixel values lies between 0 to 255. Both the test and training datasets have 785 columns, where one column represents the label and rest of the columns store the pixel values of the corresponding image.

IID partitioning involves shuffling the dataset and dividing it evenly among $N = 3$ clients so that each client receives an identical number of images.

FL Techniques: We compared our proposed approach with the original Paillier algorithm and the PFMLP approach implemented on the FedAvg algorithm.

- **FedAvg:** McMahan et al.'s [4] baseline synchronous federated learning technique involves randomly selecting a predetermined percentage of all clients for training in each round, after which the server averages the weights from those selected clients.
- **PFMLP:** Haokun Fang et al. [12] proposed an improved Paillier algorithm by introducing α as divisor ($r < \alpha$), which replaces the position of λ in the private key. The value of g in the public key is computed such that the order of g is an . In PFMLP, ciphertext and decrypted message are computed as follows:-

$$C = g^M \cdot (g^n)^r \bmod n^2$$

$$M = \frac{L(C^\alpha \bmod n^2)}{L(g^\alpha \bmod n^2)} \bmod n$$

System Settings: This framework is implemented using Pytorch [55] framework experimented on 13th Gen Intel(R) Core(TM) i9-13900 2.00 GHz with NVIDIA T400 4 GB GPU. For HE implementation Tenseal [56] library built on top of Microsoft SEAL is used. The summary of parameter values is shown in Table 5:

6.2. Performance metrics

6.2.1. Comparison in terms of learning metrics

To evaluate the performance of FL model, the system is iterated for 20 rounds over the datasets mentioned previously using Convolutional neural networks (CNN). Accuracy is the primary metric to assess a federated learning model's performance. It calculates the percentage of test data instances that are correctly classified. When comparing alternative approaches, accuracy reveals how effectively a model generalizes over data spread among clients. The average test accuracy obtained for CIFAR-10, MNIST, and Fashion MNIST datasets are 59.65%, 94.77%, and 84.92% respectively. CIFAR-10 is a large dataset where difficult to perform training, however attaining near round 60% accuracy for only 20 iterations is pretty good. The accuracy performance between baseline and proposed approach is nearly same with a minimum deviation of 1% which sounds promising for asynchronous environment. This small deviation occurs because of discarding some of the clients updates due to the expiry of timer used in proposed approach. Clients give updates at variable rates in asynchronous learning because of variety in processing capability, network conditions, or data availability. Less frequent contributors may see their updates diluted during aggregation, mainly if more active contributors dominate the updates. This imbalance can result in a global model weighted towards data from more active clients, limiting generalizability and accuracy.

6.2.2. Comparison in terms of key size

The size of cryptographic keys is crucial in influencing the system's overall security, efficiency, and scalability when implementing encryption to secure model aggregation. A comparison of key sizes between the proposed and baseline techniques is required to understand their impact on communication costs, computing overhead, and security levels. Higher security is often provided by larger key sizes, which make it more difficult for adversaries to decrypt data. However, this increased security comes with a higher computational and communication overhead. The suggested method properly balances key size selection to provide robust security and practical efficiency for federated learning. The total computation time taken for three different key sizes i.e. 128, 256, and 512 bits for MNIST, CIFAR-10, Fashion MNIST datasets can demonstrated in Figs. 7, 8, and 9 respectively. It is clear from the figures that the proposed approach reduces the total computation or training time due to batching mechanism of encrypted updates and asynchronous environment. However, compression time can add a significant amount of burden but it is handled by dealing with system idleness.

Additionally, the proposed scheme leverages Paillier homomorphic encryption, which inherently incurs a higher computational overhead owing to operations on large integers, such as modular exponentiation. This can result in increased latency and energy consumption, particularly in devices with limited processing capabilities. To address this issue, our framework adopts several optimizations. First, quantization is employed to significantly reduce the size of the encrypted messages, thereby minimizing both the number of encryption operations and communication payload. Furthermore, encryption is performed only once per parameter per iteration, which helps distribute the computational load over time and avoids bottlenecks. The system also exploits client-side idleness periods when clients are not actively engaged in training or communication to carry out encryption and compression tasks, thereby maintaining the overall training efficiency across rounds. In addition, parallelized batching of encrypted updates and an asynchronous communication model help further mitigate delays. Potential optimizations such as lightweight cryptographic libraries and hardware-based acceleration (e.g., using TPMs or dedicated cryptographic chips) are identified as promising directions to further reduce the computational burden, especially for deployment in resource-constrained IoT environments.

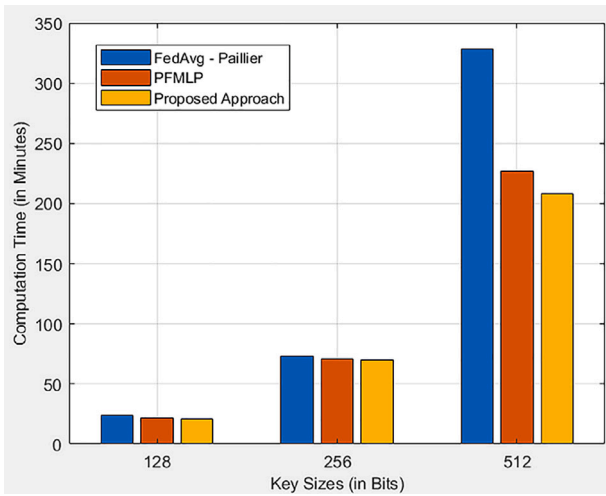


Fig. 7. Total Computation Time taken(in minutes) for MNIST Dataset in 20 iterations.

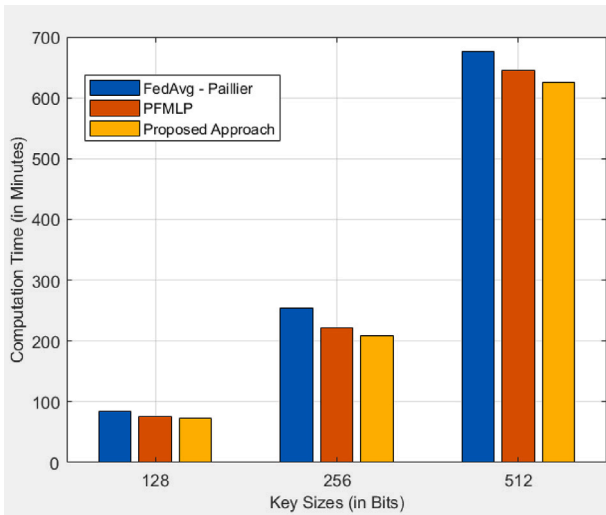


Fig. 8. Total Computation Time taken (in minutes) for CIFAR-10 Dataset in 20 iterations.

In addition to analyzing computational and communication overhead, we investigated the relationship between key size and model accuracy to understand potential trade-offs. Our experimental results revealed that varying cryptographic key lengths (128, 256, and 512 bits) did not significantly impact the predictive accuracy of the models across the CIFAR-10, MNIST, and Fashion MNIST datasets. This stability in performance can be attributed to the fact that encryption is applied solely to model updates during aggregation, not to raw training data or local model parameters. As such, the learning process at each client remains unaffected by the choice of key length. However, we acknowledge that subtle influences on model convergence due to encryption-related latency or update discards in asynchronous aggregation could emerge under more stringent network conditions or extended training rounds. These nuances present an interesting direction for future work, particularly in quantifying how encryption configurations might affect convergence speed and fairness across heterogeneous client environments.

6.2.3. Communication overhead

It starts by comparing two federated learning scenarios: one with homomorphic encryption (HE) and one without. The use of HE results in significantly longer training times and increased data transfer.

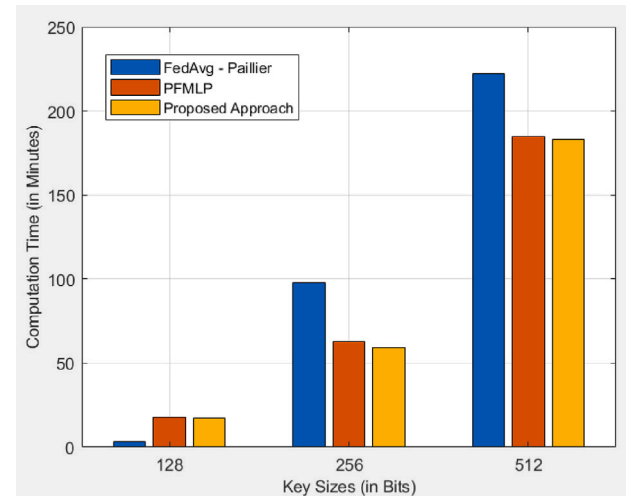


Fig. 9. Total Computation Time taken (in minutes) for Fashion MNIST Dataset in 20 iterations.

Specifically, when homomorphic encryption (HE) is enabled, the average training iteration time for FMNIST and CIFAR is 211.9 s and 2725.7 s, respectively. Compared to directly sending plaintext updates, iteration time is increased by 96X and 135X, respectively. Meanwhile, we recorded an average of 1.1 GB (6.98 MB) and 13.1 GB (85.89 MB) of data transfer each iteration between clients and the aggregator for FMNIST and CIFAR, respectively, when HE is not in use. Using HE significantly increases training time and network footprint. Complex models with multiple weights have a higher performance overhead. In case of baseline approaches, the overall data volume transferred between server and clients for 20 iterations in case of CIFAR-10, MNIST, and Fashion MNIST are 6426 GB, 929 GB, and 809 GB, whereas, in proposed approach they are 96 GB, 87 GB, and 23.2 GB respectively.

To address the overhead introduced by homomorphic encryption, the proposed framework adopts an efficient 8-bit quantization strategy prior to encryption. This approach significantly reduces the size of model updates, thereby lowering both the encryption time and the volume of data transferred across the network. Quantization compresses the floating-point model parameters into lower precision representations, which in turn results in smaller ciphertexts when encrypted with Paillier. This is especially crucial in federated environments involving complex models and resource-constrained devices, where communication and computation are major bottlenecks.

Despite the reduced precision, empirical evaluation confirms that the model maintains high performance, with only marginal accuracy deviations (within 1%) observed across the CIFAR-10, MNIST, and Fashion MNIST datasets. This confirms that the quantization scheme strikes a practical trade-off between communication efficiency and model accuracy. Additionally, by combining quantization with batching of encrypted gradients and asynchronous updates, the framework capitalizes on periods of client idleness to perform expensive computations like encryption, thereby maintaining responsiveness and scalability.

Client failures can substantially affect the convergence and performance of FL systems. These failures may be caused by hardware issues, unstable network connections, or limited computational resources. In the presence of such failures, model updates may be delayed or missed entirely, leading to stale or inconsistent global models. This is especially problematic in asynchronous FL settings, where updates are not synchronized. Moreover, as discussed above, when using HE to preserve data privacy, the computational overhead further exacerbates the challenges posed by client failures. As such, ensuring robustness against client dropouts is crucial for maintaining performance and reliability in real-world FL deployments.

7. Result discussion & Future scope

Incorporating quantization-based compression and Paillier homomorphic encryption into the asynchronous FL framework, communication costs can be significantly decreased without compromising the privacy and confidentiality of model updates. Key findings and consequences of this technique are covered in the following points.

- **Communication Efficiency:** Quantize gradient vectors before encryption significantly reduces data size, reducing communication costs. In large-scale federated learning systems, where communication is frequently a bottleneck, it is critical to decrease communication costs by compressing the gradient vectors. The fundamental constraint is the inaccuracy of quantization when transferring continuous gradient data to discrete levels. Additionally, making decryption methods more adaptable and less computationally intensive could make them suitable for the asynchronous nature of client updates. Future research could concentrate to develop adaptive quantization algorithms that vary the number of quantization levels based on real-time gradient distributions. Furthermore, investigating the integration of quantization with other compression strategies, such as low-rank approximation or gradient sparsification, could improve transmission efficiency while preserving model performance.
 - **Computational Overhead:** This includes both compression and encryption processes with quantization and Paillier homomorphic encryption. Quantization, which converts continuous gradients to discrete levels, has low overhead due to fundamental arithmetic processes. Paillier homomorphic encryption involves a large computational cost while encrypting data. This comprises resource-intensive multiplication and modular exponentiation procedures. The overhead is directly correlated with the volume of encrypted gradients and key size. Decryption, which is required for applying aggregated updates, entails identical computational procedures that add to the overall cost.
- Achieving a balance between these two processes is essential: encryption guarantees data privacy but adds significant overhead, whereas quantization lowers communication load with little computing impact. Efficiently handling both factors is critical to improve federated learning performance.
- **Scalability:** The suggested asynchronous FL framework has robust scalability, particularly when dealing with increasing number of clients without significantly reducing performance. The framework's asynchronous structure enables clients to update the model at separate times, decreasing the impact of stragglers and increasing overall efficiency. The approach efficiently lowers communication overhead when combined with quantization-based compression, which is important in large-scale, bandwidth-constrained applications like mobile devices or IoT networks. However, scalability poses particular challenges in an asynchronous environment. With an increase in the number of clients, the accumulation of quantization errors across staggered updates may cause inconsistencies in the aggregated model and perhaps reduce accuracy.

Paillier homomorphic encryption computational complexity scales with the number of clients, making it difficult for devices with insufficient processing capacity to perform. It could worsen synchronization, making it difficult to strike a balance between timely updates and model accuracy.

It is recommended that future studies investigate adaptive strategies designed for asynchronous situations. Model consistency might be preserved by dynamically adjusting quantization levels in response to client activity and data distribution. Large-scale asynchronous update difficulty could be managed and bottlenecks could be minimized with the use of hierarchical or clustered aggregation techniques. Additionally, in real-world asynchronous

federated learning settings, improving scalability while protecting data privacy would need the development of more effective encryption protocols or hybrid techniques.

- **Fairness Issues:** Client participation in an asynchronous FL environment causes fairness challenges. The global model may be disproportionately influenced by clients who update frequently, producing biased results. Data heterogeneity worsens this issue since models trained on dominant clients would not function well on lagging clients. Clients with different computational resources and network latency also lead to unequal influence: clients with more resources tend to distort the model's behavior. Furthermore, stale updates from some clients can result from asynchronous aggregation, which lowers the model's accuracy and fairness for those who are lagging. To solve these problems, fairness-aware optimization, adaptive strategies, and careful aggregation technique design are needed to balance the contributions of each client and ensure equal model performance across different client populations.
- **Real-world Deployment Scenarios:** While the proposed approach has been extensively validated using benchmark datasets such as MNIST, CIFAR-10, and Fashion MNIST, real-world deployment remains a critical step for evaluating system robustness and generalizability. Practical scenarios such as FL in healthcare (e.g., collaborative training across hospitals without sharing sensitive patient data) or smart environments (e.g., IoT-based energy monitoring systems) stand to benefit from the proposed privacy-preserving, communication-efficient framework. These settings often involve clients with intermittent availability and heterogeneous computational capacities, making the asynchronous structure of our framework particularly advantageous. Future work will focus on deploying the proposed scheme in such real-world federated environments to analyze factors like latency under fluctuating network conditions, resource constraints on edge devices, and policy compliance in privacy-sensitive domains. This real-world validation is essential to understand system performance beyond controlled experiments and drive further refinements for practical adoption.

8. Conclusion

The federated learning system presented in this study is specifically made for asynchronous contexts. It incorporates quantization-based compression and improved Paillier homomorphic encryption to solve important privacy, security, and communication efficiency issues. Our technique, which employs homomorphic encryption, ensures that important model changes are safeguarded throughout transmission, even in the presence of a central server that accumulates gradients without direct data access. By including quantization, the architecture becomes more appropriate for distributed systems with bandwidth limitations, as it drastically lowers the communication overhead. The framework's asynchronous nature enables clients to function independently, delivering updates at various times while maintaining the integrity of the learning process via a timestamp system that synchronizes aggregate. This adaptability is critical in real-world circumstances where clients have different processing resources and network conditions. The experimental results show that the suggested framework achieves excellent model performance without sacrificing data security by carefully balancing the trade-offs between accuracy, privacy, and communication efficiency.

Overall, this study presents a comprehensive solution for federated learning in decentralized and asynchronous environments, paving the way for safe, scalable, and efficient collaborative machine learning across varied and remote data sources. Future research could examine more quantization process optimizations, improve scalability to support more clients, and assess the framework's effectiveness on other datasets and network scenarios.

CRedit authorship contribution statement

Mansi Gupta: Writing – review & editing, Writing – original draft, Methodology, Conceptualization. **Mohit Kumar:** Writing – review & editing, Visualization, Validation, Supervision. **Renu Dhir:** Writing – review & editing, Supervision, Formal analysis.

Funding information

The authors did not receive any grants or funds to support this research work.

The article did not involve human and/or animals' participants.

The paper reflects the authors' own research and analysis in a truthful and complete manner. Authors ensure that the work is original and has not been published elsewhere.

Declaration of competing interest

The authors whose names are given in this article certify that they have no affiliations with or involvement in any organization or entity with any financial interest (such as honoraria; educational grants; participation in speaker's bureaus; membership, employment, consultancies, stock ownership, or other equity interest; and expert testimony or patent-licensing arrangements), or non-financial interest (such as personal or professional relationships, affiliations, knowledge or beliefs) in the subject matter or materials discussed in this manuscript. Currently there is no concurrent submission of this article by any of authors.

Data availability

Data will be made available on request.

References

- [1] Korteling JH, van de Boer-Visschedijk GC, Blankendaal RA, Boonekamp RC, Eikelboom AR. Human-versus artificial intelligence. *Front Artif Intell* 2021;4:622364.
- [2] Herath H, Mittal M. Adoption of artificial intelligence in smart cities: A comprehensive review. *Int J Inf Manag Data Insights* 2022;2(1):100076.
- [3] Truong N, Sun K, Wang S, Guitton F, Guo Y. Privacy preservation in federated learning: An insightful survey from the GDPR perspective. *Comput Secur* 2021;110:102402.
- [4] McMahan B, Moore E, Ramage D, Hampson S, y Arcas BA. Communication-efficient learning of deep networks from decentralized data. In: *Artificial intelligence and statistics*. PMLR; 2017, p. 1273–82.
- [5] AbdulRahman S, Tout H, Ould-Slimane H, Mourad A, Talhi C, Guizani M. A survey on federated learning: The journey from centralized to distributed on-site learning and beyond. *IEEE Internet Things J* 2020;8(7):5476–97.
- [6] Agrawal S, Sarkar S, Aouedi O, Yenduri G, Piamrat K, Alazab M, Bhat-tacharya S, Maddikunta PKR, Gadekallu TR. Federated learning for intrusion detection system: Concepts, challenges and future directions. *Comput Commun* 2022;195:346–61.
- [7] Ku H, Susilo W, Zhang Y, Liu W, Zhang M. Privacy-preserving federated learning in medical diagnosis with homomorphic re-encryption. *Comput Stand Interfaces* 2022;80:103583.
- [8] Ma J, Naas S-A, Sigg S, Lyu X. Privacy-preserving federated learning based on multi-key homomorphic encryption. *Int J Intell Syst* 2022;37(9):5880–901.
- [9] Park J, Lim H. Privacy-preserving federated learning using homomorphic encryption. *Appl Sci* 2022;12(2):734.
- [10] Wibawa F, Catak FO, Kuzlu M, Sarp S, Cali U. Homomorphic encryption and federated learning based privacy-preserving cnn training: Covid-19 detection use-case. In: *Proceedings of the 2022 European interdisciplinary cybersecurity conference*. 2022, p. 85–90.
- [11] Zhang C, Li S, Xia J, Wang W, Yan F, Liu Y. {BatchCrypt}: Efficient homomorphic encryption for {cross-silo} federated learning. In: *2020 USENIX annual technical conference*. USENIX ATC 20, 2020, p. 493–506.
- [12] Fang H, Qian Q. Privacy preserving machine learning with homomorphic encryption and federated learning. *Futur Internet* 2021;13(4):94.
- [13] Zhang L, Xu J, Vijayakumar P, Sharma PK, Ghosh U. Homomorphic encryption-based privacy-preserving federated learning in IoT-enabled healthcare system. *IEEE Trans Netw Sci Eng* 2022;10(5):2864–80.
- [14] Hijazi NM, Aloqaily M, Guizani M, Ouni B, Karray F. Secure federated learning with fully homomorphic encryption for iot communications. *IEEE Internet Things J* 2023.
- [15] Li G, Hu Y, Zhang M, Liu J, Yin Q, Peng Y, Dou D. FedHiSyn: A hierarchical synchronous federated learning framework for resource and data heterogeneity. In: *Proceedings of the 51st international conference on parallel processing*. 2022, p. 1–11.
- [16] Sarikaya Y, Ercetin O. Motivating workers in federated learning: A stackelberg game perspective. *IEEE Netw Lett* 2019;2(1):23–7.
- [17] Li T, Sahu AK, Zaheer M, Sanjabi M, Talwalkar A, Smith V. Federated optimization in heterogeneous networks. *Proc Mach Learn Syst* 2020;2:429–50.
- [18] Wang H, Yurochkin M, Sun Y, Papailiopoulos D, Khazaeni Y. Federated learning with matched averaging. 2020, arXiv preprint arXiv:2002.06440.
- [19] Lu X, Liao Y, Lio P, Hui P. Privacy-preserving asynchronous federated learning mechanism for edge network computing. *IEEE Access* 2020;8:48970–81.
- [20] Chen T, Jin X, Sun Y, Yin W. Vaf: a method of vertical asynchronous federated learning. 2020, arXiv preprint arXiv:2007.06081.
- [21] Li M, Chen Y, Wang Y, Pan Y. Efficient asynchronous vertical federated learning via gradient prediction and double-end sparse compression. In: *2020 16th international conference on control, automation, robotics and vision. ICARCV, IEEE; 2020, p. 291–6*.
- [22] Xie C, Koyejo S, Gupta I. Asynchronous federated optimization. 2019, arXiv preprint arXiv:1903.03934.
- [23] Melis L, Song C, De Cristofaro E, Shmatikov V. Exploiting unintended feature leakage in collaborative learning. In: *2019 IEEE symposium on security and privacy. SP, IEEE; 2019, p. 691–706*.
- [24] Mothukuri V, Parizi RM, Pouriyeh S, Huang Y, Dehghantanha A, Srivastava G. A survey on security and privacy of federated learning. *Future Gener Comput Syst* 2021;115:619–40.
- [25] Sharma A, Marchang N. A review on client-server attacks and defenses in federated learning. *Comput Secur* 2024;103801.
- [26] Muñoz-González L, Biggio B, Demontis A, Paudice A, Wongrassamee V, Lupu EC, Roli F. Towards poisoning of deep learning algorithms with back-gradient optimization. In: *Proceedings of the 10th ACM workshop on artificial intelligence and security*. 2017, p. 27–38.
- [27] Biggio B, Nelson B, Laskov P. Poisoning attacks against support vector machines. 2012, arXiv preprint arXiv:1206.6389.
- [28] Mondal A, Virk H, Gupta D. Beas: Blockchain enabled asynchronous & secure federated machine learning. 2022, arXiv preprint arXiv:2202.02817.
- [29] Feng L, Zhao Y, Guo S, Qiu X, Li W, Yu P. BAF: A blockchain-based asynchronous federated learning framework. *IEEE Trans Comput* 2021;71(5):1092–103.
- [30] Xu C, Ge J, Deng Y, Gao L, Zhang M, Li Y, Zhou W, Zheng X. BASS: A blockchain-based asynchronous signsgd architecture for efficient and secure federated learning. *IEEE Trans Dependable Secur Comput* 2024.
- [31] Zhang J, Li S. Blockchain-empowered vehicular intelligence: A perspective of asynchronous federated learning. *IEEE Internet Things Mag* 2024;7(1):74–80.
- [32] Chen Z, Cui H, Wu E, Yu X. Dynamic asynchronous anti poisoning federated deep learning with blockchain-based reputation-aware solutions. *Sensors* 2022;22(2):684.
- [33] Dwork C. Differential privacy. In: *International colloquium on automata, languages, and programming*. Springer; 2006, p. 1–12.
- [34] Canetti R, Feige U, Goldreich O, Naor M. Adaptively secure multi-party computation. In: *Proceedings of the twenty-eighth annual ACM symposium on theory of computing*. 1996, p. 639–48.
- [35] Rivest RL, Adleman L, Dertouzos ML, et al. On data banks and privacy homomorphisms. *Found Secur Comput* 1978;4(11):169–80.
- [36] Munjal K, Bhatia R. A systematic review of homomorphic encryption and its contributions in healthcare industry. *Complex & Intell Syst* 2023;9(4):3759–86.
- [37] Acar A, Aksu H, Uluagac AS, Conti M. A survey on homomorphic encryption schemes: Theory and implementation. *ACM Comput Surv (Csur)* 2018;51(4):1–35.
- [38] Rivest RL, Shamir A, Adleman L. A method for obtaining digital signatures and public-key cryptosystems. *Commun ACM* 1978;21(2):120–6.
- [39] Goldwasser S, Micali S. Probabilistic encryption & how to play mental poker keeping secret all partial information. In: *Providing sound foundations for cryptography: on the work of shafi goldwasser and silvio micali*. 2019, p. 173–201.
- [40] ElGamal T. A public key cryptosystem and a signature scheme based on discrete logarithms. *IEEE Trans Inform Theory* 1985;31(4):469–72.
- [41] Paillier P. Public-key cryptosystems based on composite degree residuosity classes. In: *International conference on the theory and applications of cryptographic techniques*. Springer; 1999, p. 223–38.
- [42] Sander T, Young A, Yung M. Non-interactive cryptocomputing for nc/sup 1. In: *40th annual symposium on foundations of computer science (cat. no. 99CB37039)*. IEEE; 1999, p. 554–66.
- [43] Ishai Y, Paskin A. Evaluating branching programs on encrypted data. In: *Theory of cryptography conference*. Springer; 2007, p. 575–94.

- [44] Boneh D, Goh E-J, Nissim K. Evaluating 2-DNF formulas on ciphertexts. In: Theory of cryptography: second theory of cryptography conference, TCC 2005, Cambridge, MA, USA, February 10-12, 2005. proceedings 2. Springer; 2005, p. 325–41.
- [45] Gentry C. A fully homomorphic encryption scheme. Stanford university; 2009.
- [46] Van Dijk M, Gentry C, Halevi S, Vaikuntanathan V. Fully homomorphic encryption over the integers. In: Advances in cryptology–EUROCRYPT 2010: 29th annual international conference on the theory and applications of cryptographic techniques, French riviera, May 30–June 3, 2010. proceedings 29. Springer; 2010, p. 24–43.
- [47] Brakerski Z, Vaikuntanathan V. Fully homomorphic encryption from ring-LWE and security for key dependent messages. In: Annual cryptology conference. Springer; 2011, p. 505–24.
- [48] López-Alt A, Tromer E, Vaikuntanathan V. On-the-fly multiparty computation on the cloud via multikey fully homomorphic encryption. In: Proceedings of the forty-fourth annual ACM symposium on theory of computing. 2012, p. 1219–34.
- [49] Aji AF, Heafield K. Sparse communication for distributed gradient descent. In: Proceedings of the 2017 conference on empirical methods in natural language processing. EMNLP, Association for Computational Linguistics; 2017, p. 440–5.
- [50] Alistarh D, Grubic D, Li J, Tomioka R, Vojnovic M. QSGD: Communication-efficient SGD via gradient quantization and encoding. In: Advances in neural information processing systems (neurIPS). Vol. 30, 2017.
- [51] Beutel DJ, Topal T, Mathur A, Qiu X, Fernandez-Marques J, Gao Y, Sani L, Li KH, Parcollet T, de Gusmão PPB, et al. Flower: A friendly federated learning research framework. 2020, arXiv preprint [arXiv:2007.14390](https://arxiv.org/abs/2007.14390).
- [52] Krizhevsky A, Hinton G, et al. Learning multiple layers of features from tiny images. 2009.
- [53] Yann Lecun CC, Burges CJ. MNIST handwritten digit database, yann lecut, corinna cortes and chris burges — [yann.lecun.com](http://yann.lecun.com/exdb/mnist/). 2010, [Accessed 01 May 2024] <http://yann.lecun.com/exdb/mnist/>.
- [54] Fashion MNIST — [kaggle.com](https://www.kaggle.com/datasets/zalando-research/fashionmnist). 2017, [Accessed 23 August 2024] <https://www.kaggle.com/datasets/zalando-research/fashionmnist>.
- [55] Imambi S, Prakash KB, Kanagachidambaresan G. Pytorch. Program TensorFlow: Solut Edge Comput Appl 2021;87–104.
- [56] Benaissa A, Retiat B, Cebere B, Belfedhal AE. Tenseal: A library for encrypted tensor operations using homomorphic encryption. 2021, arXiv preprint [arXiv: 2104.03152](https://arxiv.org/abs/2104.03152).